Assignment 5
Advanced JavaScript and DOM Manipulation

PixelForge - an interactive gaming website continuing Assignments 3 and 4.

Team	PixelForge
Group	SE-2501
Members	Marat Nurdaulet; Kadyr Tlektes; Suinalin Azamat
Prepared	23 October 2026

Objective

Create a useful, engaging gaming interface that demonstrates DOM selection, content and style updates, attributes, events, callbacks, switch statements, objects, arrays, loops, higher-order functions, sound and animation.

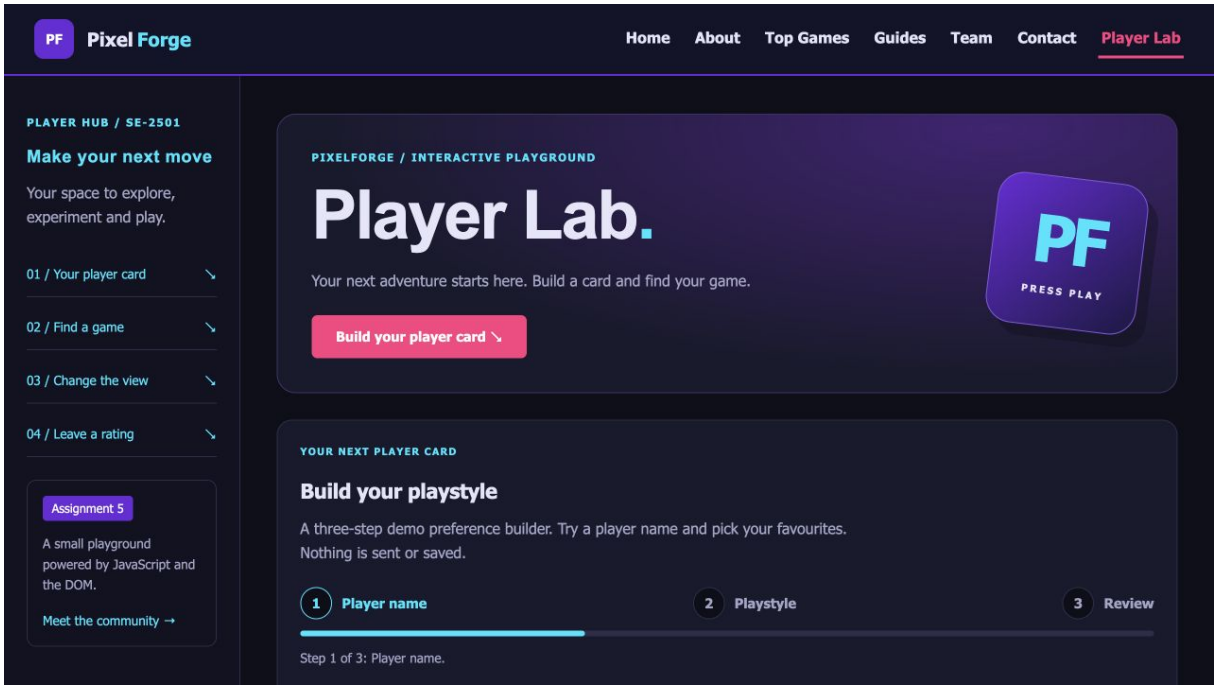


Figure 1. Player Lab running in the browser at 1280px. All webpage captures use the actual implemented project.

Website: https://nurda07.github.io/Web-assignment/
Player Lab: https://nurda07.github.io/Web-assignment/play.html
Repository: https://github.com/NURDA07/Web-assignment

Source excerpts below are rendered directly from the submitted files, with original line numbers. Webpage screenshots are browser captures; some are cropped to the feature under discussion.

1. DOM selection and dynamic styling

1. Select the five star controls with querySelectorAll. Each button stores its value in a data-rating attribute.
2. On a click, call selectRating(value) and loop through the selected elements.
3. Change the star color directly through style.color, update classes, aria-checked and keyboard tabindex, then replace the feedback text.

assignment3/js/play.js | lines 89–103

```
89    // Accessible custom radio buttons: arrows change the rating and focus
        together.
90    const stars = Array.from(document.querySelectorAll("#rating-stars [data-
        rating]"));
91    const ratingStatus = document.getElementById("rating-status");
92
93    function selectRating(value) {
94      stars.forEach((star) => {
95        const starValue = Number(star.dataset.rating);
96        star.classList.toggle("is-filled", starValue <= value);
97        // Direct DOM styling: the selected rating lights up this star and all
        before it.
98        star.style.color = starValue <= value ? "#ffd36b" : "#777797";
99        star.setAttribute("aria-checked", String(starValue === value));
100       star.tabIndex = starValue === value ? 0 : -1;
101     });
102     ratingStatus.textContent = `Your rating: ${value} / 5. Thanks for
        playing!`;
103   }
```

Actual source excerpt, rendered with its original line numbers.



Figure 2. A five-star selection changes the visible color and feedback. Keyboard ArrowRight from four stars also selected five.

Result: content, attributes and visual styles change immediately without reloading the page. The custom radio group has a single keyboard entry point and announces the current rating.

2. Manipulating image attributes

Chosen option: a thumbnail gallery.

1. Store each scene in an object with image, title, alternative text and caption.
2. Attach a click listener to each thumbnail and retrieve the chosen scene by data-image.
3. Set the large image src and alt, update the caption, and mark the selected thumbnail with aria-pressed.

assignment3/js/play.js | lines 128–142

```
128    const featuredImage = document.getElementById("featured-image");
129    const thumbnails = document.querySelectorAll("#gallery-thumbnails [data-
         image]");
130
131    thumbnails.forEach((button) => button.addEventListener("click", () => {
132      const scene = scenes[Number(button.dataset.image)];
133      featuredImage.src = scene.image;
134      featuredImage.alt = scene.alt;
135      document.getElementById("featured-title").textContent = scene.title;
136      document.getElementById("featured-caption").textContent = scene.caption;
137      document.getElementById("gallery-status").textContent = `Featured image
         changed to ${scene.title}.`;
138      thumbnails.forEach((thumbnail) => {
139        const selected = thumbnail === button;
140        thumbnail.classList.toggle("is-selected", selected);
141        thumbnail.setAttribute("aria-pressed", String(selected));
142      });
```

Actual source excerpt, rendered with its original line numbers.

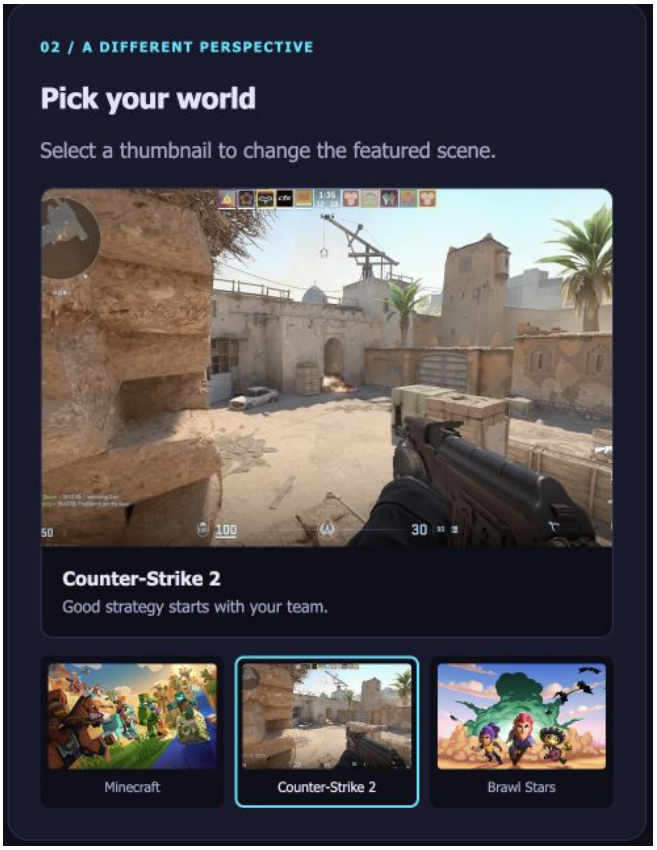


Figure 3. Selecting Counter-Strike 2 changes the large image, its caption and thumbnail border.

Verification: the image src became images/counterstrike2.png, the alternative text described that image, and exactly one thumbnail remained selected.

3. Objects, arrays, loops and filtering

1. Keep the game collection in an array of objects. Each object has a getRating() method using this.rating.
2. Use switch(category) to select a category, then filter() and map() to prepare the visible cards.
3. Append cards with a for...of loop. Update the result count and selected button after every category click.

assignment3/js/play.js | lines 11–17

```
11    const games = [
12      {
13        title: "Counter-Strike 2", category: "fps", platform: "PC", rating: 9.5,
14        image: "images/counterstrike2.png", alt: "Counter-Strike 2 tactical
        shooter artwork",
15        description: "Think together. Hold the angle. Make every round count.",
16        getRating() { return `${this.rating.toFixed(1)} / 10`; }
17      },
```

Actual source excerpt, rendered with its original line numbers.

assignment3/js/play.js | lines 37–47

```
37    function renderGames(category) {
38      let categoryName;
39      switch (category) {
40        case "fps": categoryName = "Tactical FPS"; break;
41        case "sandbox": categoryName = "Sandbox"; break;
42        case "arena": categoryName = "Arena"; break;
43        default: category = "all"; categoryName = "All games";
44      }
45
46      const visibleGames = games.filter((game) => category === "all" ||
        game.category === category);
47      const cards = visibleGames.map((game) => {
```

Actual source excerpt, rendered with its original line numbers.

assignment3/js/play.js | lines 71–74

```
71      catalog.replaceChildren();
72      for (const card of cards) {
73        catalog.append(card);
74      }
```

Actual source excerpt, rendered with its original line numbers.

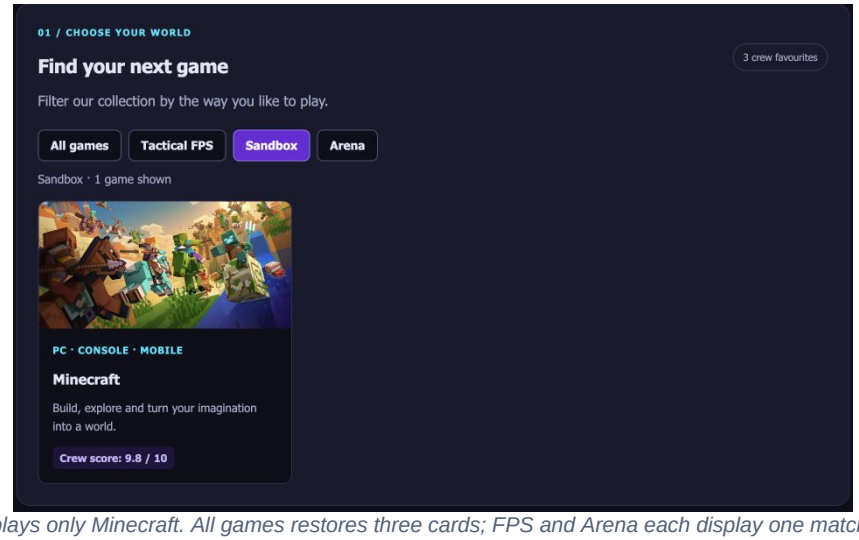


Figure 4. Sandbox displays only Minecraft. All games restores three cards; FPS and Arena each display one matching game.

filter, map and forEach are higher-order functions because they receive callbacks. Card content is inserted with textContent rather than parsing user text as HTML.

4. Button and keyboard events

1. Load navigation.js on every HTML page with defer.
2. Listen for keydown within each nav and find the currently focused link among visible links.
3. Use switch(event.key) to move focus with the arrow keys and wrap around the group. Home and End select the first and last links. Prevent scrolling only for handled keys; Tab and Enter keep their normal behavior.

assignment3/js/navigation.js | lines 9–34

```
 9      if (currentIndex < 0) return;
10
11      let nextIndex;
12      switch (event.key) {
13        case 'ArrowRight':
14        case 'ArrowDown':
15          nextIndex = (currentIndex + 1) % links.length;
16          break;
17        case 'ArrowLeft':
18        case 'ArrowUp':
19          nextIndex = (currentIndex - 1 + links.length) % links.length;
20          break;
21        case 'Home':
22          nextIndex = 0;
23          break;
24        case 'End':
25          nextIndex = links.length - 1;
26          break;
27        default:
28          return;
29      }
30      event.preventDefault();
31      links[nextIndex].focus();
32    });
33  });
```

Actual source excerpt, rendered with its original line numbers.

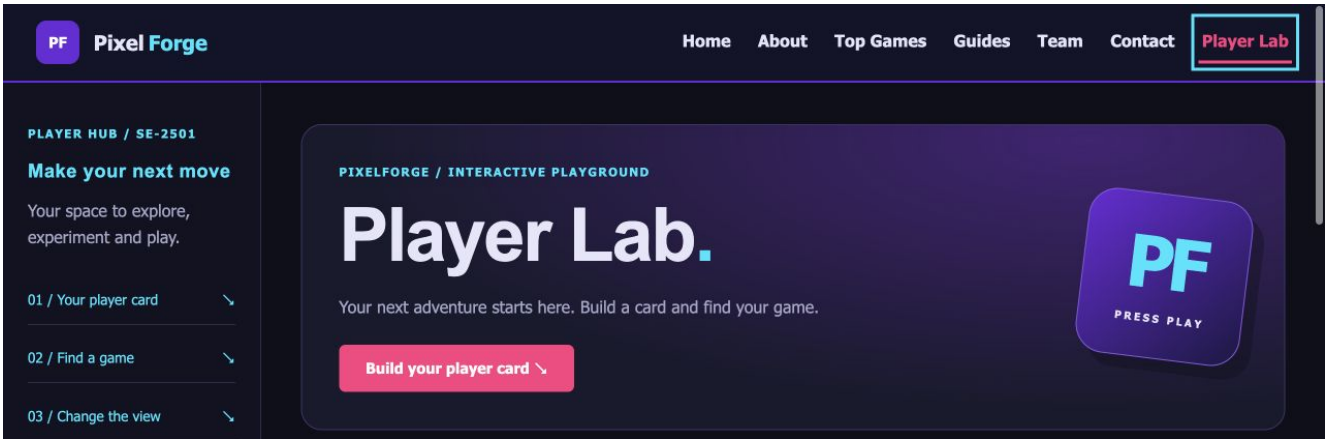


Figure 5. End moves keyboard focus to Player Lab. ArrowRight from Home was also verified to focus About.

Other click listeners control filters, gallery thumbnails, stars, the profile steps, Reset, the quest and the sound. No inline event handlers are used.

5. Callback-based multi-step form

Chosen option: a three-step form that keeps its input state.
1. Generate the Player name, Playstyle and Review panels from arrays. Store choices in the profileState object.
2. Validate only the visible step. A missing name, platform or genre keeps the user on that step and moves focus to the first error.
3. Call changeStep(nextIndex, onStepChange). The callback updates visibility, progress, button labels and focus.

assignment3/js/player-profile.js | lines 191–208

```
191   // A higher-order function: the callback receives the new step and renders
        it.
192   function changeStep(nextIndex, callback) {
193     profileState.step = Math.max(0, Math.min(nextIndex, stepNames.length -
        1));
194     profileState.completed = false;
195     callback(profileState.step);
196   }
197
198   nickname.addEventListener("input", () => {
199     profileState.nickname = nickname.value;
200     if (nicknameError.textContent) setError(nickname, nicknameError, "");
201   });
202   platform.addEventListener("change", () => {
203     profileState.platform = platform.value;
204     if (platform.value) setError(platform, platformError, "");
205   });
206   genreInputs.forEach(input => {
207     input.addEventListener("change", () => {
208       profileState.genres = genreInputs.filter(option =>
        option.checked).map(option => option.value);
```

Actual source excerpt, rendered with its original line numbers.

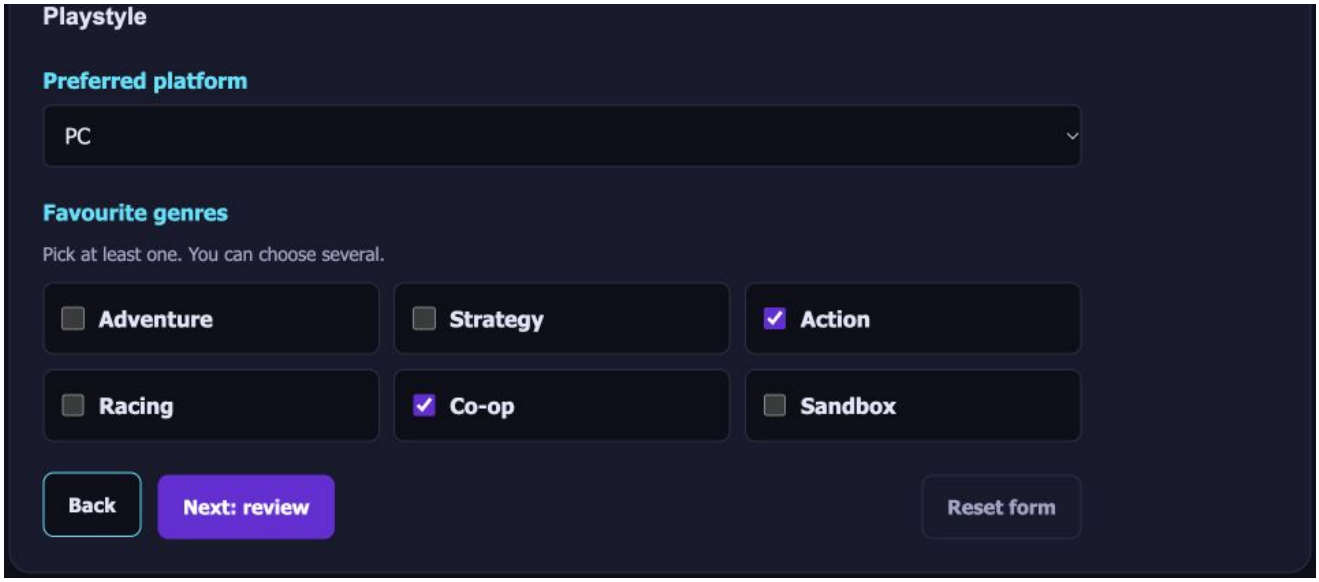


Figure 6. Returning from Review preserves PC plus the Action and Co-op selections. Only the Playstyle panel is active.

The callback is passed as a value rather than called immediately. changeStep updates the state, then invokes that function with the new index. This separates navigation logic from rendering.

6. Review, completion and reset

1. Build the Review summary from profileState.summary(), using DOM elements and textContent.
2. On the final submit, display a demo completion message and disable repeated completion.
3. Back keeps previous inputs. Reset calls form.reset(), resets the state object, clears errors and returns to step one.

assignment3/js/player-profile.js | lines 231–242

```
231    backButton.addEventListener("click", () => changeStep(profileState.step - 1,
           onStepChange));
232    resetButton.addEventListener("click", () => {
233      form.reset();
234      profileState.reset();
235      setError(nickname, nicknameError, "");
236      setError(platform, platformError, "");
237      genresError.textContent = "";
238      genreInputs.forEach(input => input.removeAttribute("aria-invalid"));
239      completion.textContent = "";
240      changeStep(0, onStepChange);
241      status.textContent = "Form reset. Step 1 of 3: Player name.";
242      nickname.focus();
```

Actual source excerpt, rendered with its original line numbers.

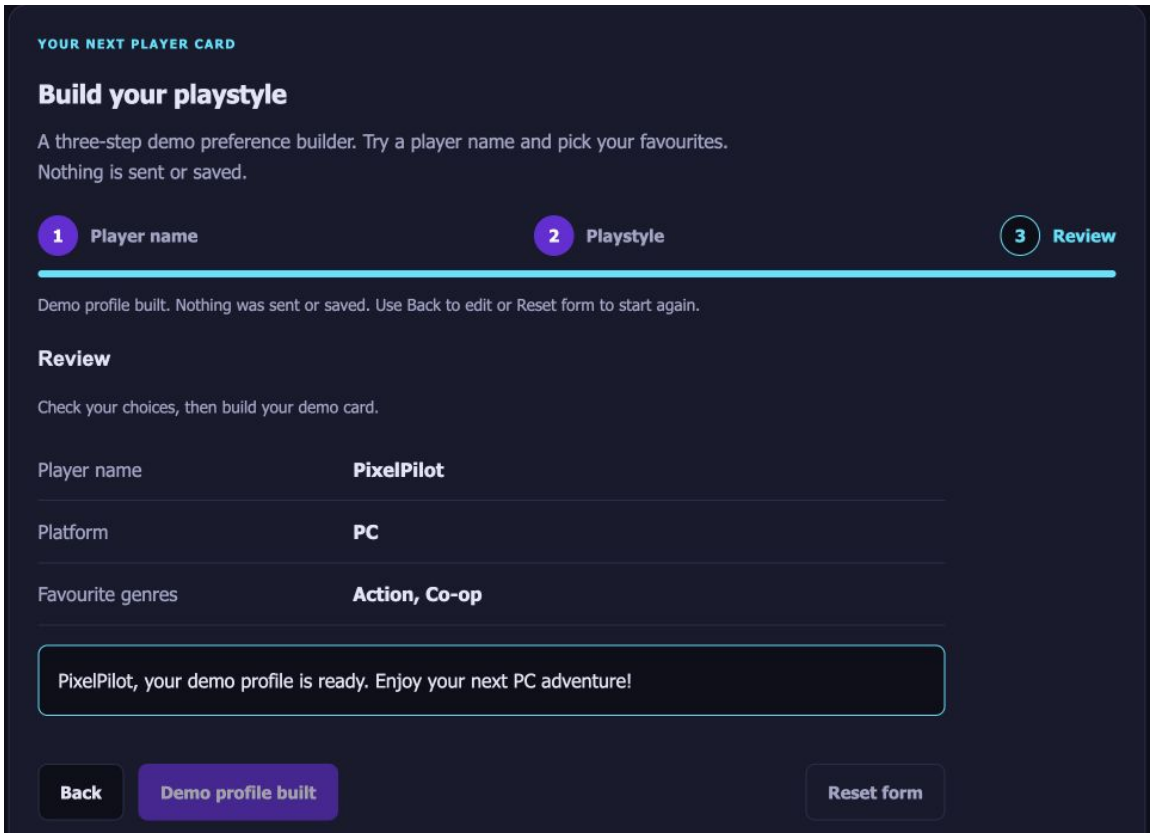


Figure 7. The completed demo shows PixelPilot, PC, Action and Co-op. No account or stored profile is created.

Verification: empty steps were blocked; Back preserved choices; completion displayed the correct summary; Reset cleared the name, platform and all genre checkboxes and returned to step one. All form actions stayed on the current page.

7. User-triggered sound effects

1. Create an AudioContext only after the user presses Play victory sound. Resume it when needed.
2. Generate a short original chime from three sine-wave notes, with a gain envelope for each note.
3. Keep soundPlaying true and disable the control during playback. Note end callbacks disconnect audio nodes and re-enable the control after the final note.

assignment3/js/play.js | lines 185–203

```
185    soundButton.addEventListener("click", async () => {
186      if (soundPlaying || !AudioContextClass) return;
187      soundPlaying = true;
188      soundButton.disabled = true;
189      try {
190        audioContext = audioContext || new AudioContextClass();
191        if (audioContext.state === "suspended") await audioContext.resume();
192        if (audioContext.state !== "running") throw new Error("Audio is not
         available");
193        const frequencies = [523.25, 659.25, 783.99];
194        let remainingNotes = frequencies.length;
195        frequencies.forEach((frequency, index) => {
196          const oscillator = audioContext.createOscillator();
197          const volume = audioContext.createGain();
198          const start = audioContext.currentTime + index * .18;
199          oscillator.type = "sine";
200          oscillator.frequency.value = frequency;
201          volume.gain.setValueAtTime(0, start);
202          volume.gain.linearRampToValueAtTime(.08, start + .015);
203          volume.gain.exponentialRampToValueAtTime(.001, start + .16);
```

Actual source excerpt, rendered with its original line numbers.

assignment3/js/play.js | lines 206–217

```
206          oscillator.onended = () => {
207            oscillator.disconnect();
208            volume.disconnect();
209            remainingNotes -= 1;
210            if (remainingNotes === 0) {
211              soundPlaying = false;
212              soundButton.disabled = false;
213              soundStatus.textContent = "Victory sound finished. You can play it
         again.";
214            }
215          };
216          oscillator.start(start);
217          oscillator.stop(start + .17);
```

Actual source excerpt, rendered with its original line numbers.

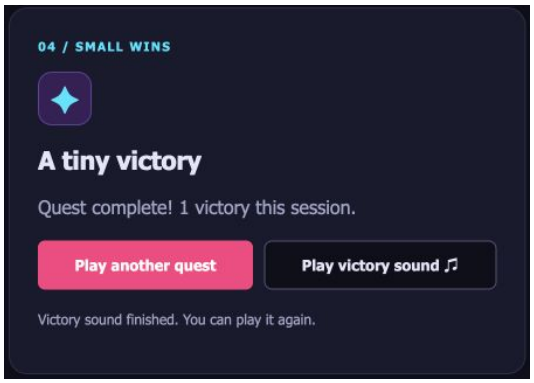


Figure 8. After playback the message confirms completion and the sound button is available again.

Verification: the playing message and disabled button appeared, then the finished message and enabled button returned. The guard prevents repeated clicks from overlapping chimes. Unsupported audio or a failed start receives visible feedback.

8. Event-triggered animation

1. Launch quest starts an animation and marks the card busy.
2. animate() changes translateY and scale, then the handler waits for animation.finished.
3. Increment the victory count, update the status and re-enable the button. A cancelled or unsupported animation still completes the demo.
4. If prefers-reduced-motion is enabled, skip movement and show the result immediately. The profile CSS also disables motion under that preference.

assignment3/js/play.js | lines 154–171

```
154   questButton.addEventListener("click", async () => {
155     questButton.disabled = true;
156     questCard.setAttribute("aria-busy", "true");
157     questStatus.textContent = "Quest in progress…";
158     if (!reducedMotion.matches && typeof questCard.animate === "function") {
159       const animation = questCard.animate([
160         { transform: "translateY(0) scale(1)" },
161         { transform: "translateY(-8px) scale(1.025)", offset: .45 },
162         { transform: "translateY(0) scale(1)" }
163       ], { duration: 650, easing: "ease-in-out" });
164       try { await animation.finished; } catch { /* A cancelled animation still
        completes the demo quest. */ }
165     }
166     completedQuests += 1;
167     questStatus.textContent = `Quest complete! ${completedQuests}
      ${completedQuests === 1 ? "victory" : "victories"} this session.`;
168     questButton.textContent = "Play another quest";
169     questCard.removeAttribute("aria-busy");
170     questButton.disabled = false;
171   });
```

Actual source excerpt, rendered with its original line numbers.

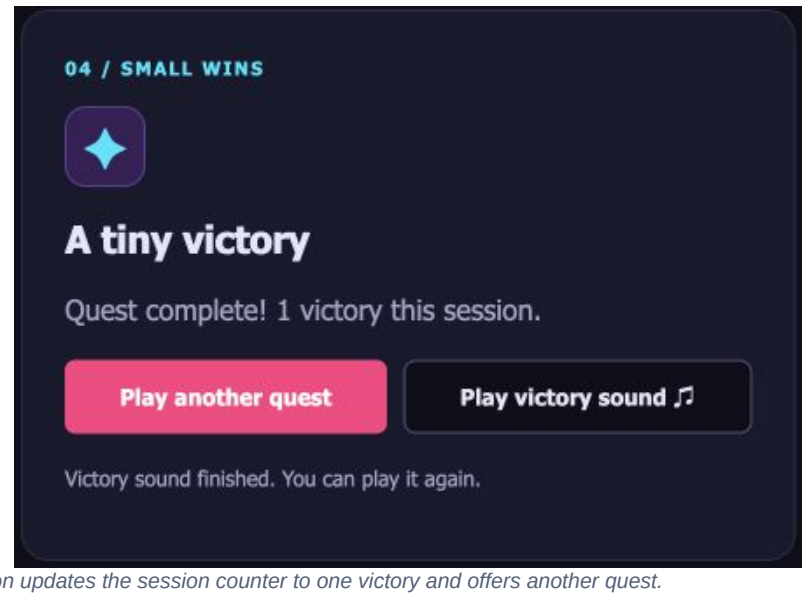


Figure 9. Quest completion updates the session counter to one victory and offers another quest.

Additional transitions accompany catalog filtering, image selection and profile step changes. The game theme gives these interactions a purpose rather than adding unrelated motion.

9. Structure and responsive interface

1. Continue the existing PixelForge project and keep the common dark theme.
2. Separate structure, presentation and behavior into HTML, external CSS and external JavaScript. Earlier Home and Media Queries style blocks were moved into their own CSS files.
3. Add Player Lab links to the existing pages and shared navigation behavior throughout the site.

assignment3/play.html | lines 7–13

```
 7    <title>PixelForge – Player Lab</title>
 8    <link rel="stylesheet" href="css/style.css">
 9    <link rel="stylesheet" href="css/play.css">
10    <link rel="stylesheet" href="css/player-profile.css">
11    <script src="js/navigation.js" defer></script>
12    <script src="js/play.js" defer></script>
13    <script src="js/player-profile.js" defer></script>
```

Actual source excerpt, rendered with its original line numbers.

File / folder	Purpose
play.html	Player Lab structure and accessible controls
css/play.css; css/player-profile.css	Layout, colors, responsive rules and transitions
js/play.js	Catalog, rating, gallery, sound and quest
js/player-profile.js	Profile state, validation, callbacks and reset
js/navigation.js	Keyboard navigation shared by every page

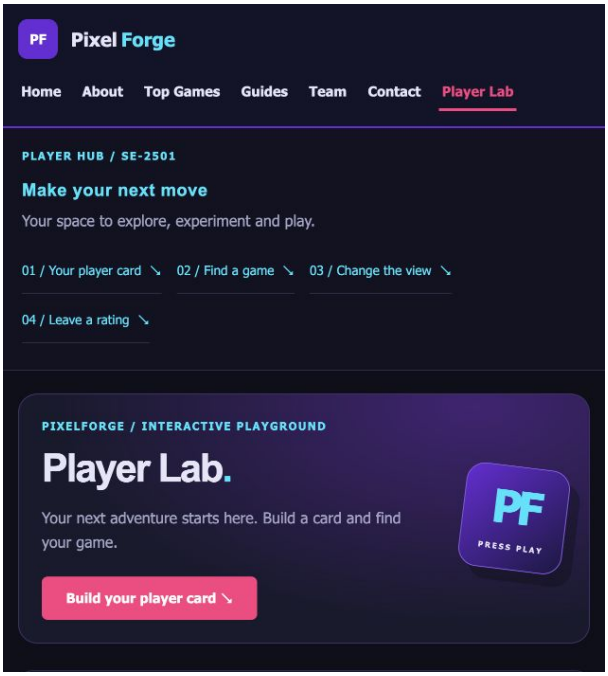


Figure 10. The 636px layout stacks navigation, sidebar and main content. No horizontal overflow was observed at 636px or 1280px.

The project has no build step or third-party JavaScript dependency for Player Lab. Existing Bootstrap pages still load Bootstrap from a CDN. All images used by the new page are included locally.

10. Verification and deployment

Check	Observed result
Catalog: FPS / Sandbox / Arena / All	1 / 1 / 1 / 3 matching game cards
Gallery selection	src, alt, caption and selected state update
Stars and keyboard	Click 4 then ArrowRight selects 5; color and aria state update
Navigation keyboard	ArrowRight: Home to About; End: last link
Profile invalid steps	Missing name/platform/genres show errors
Profile Back and completion	Choices preserved; correct review and completion
Profile Reset	All inputs clear; returns to step one
Quest and sound	Busy controls disable, then re-enable on completion
Layout / console	No horizontal overflow at 1280px and 636px; no JS console errors
Static source audit	8 HTML files, 161 local links, 17 fragments; no missing targets or duplicate IDs
Separation / syntax	No inline styles or scripts; all 4 JS files pass node --check

Deployment steps

1. Commit the project, README, defense notes and this report to main in the shared GitHub repository.
2. The existing GitHub Actions workflow uploads assignment3/ and deploys it to GitHub Pages.
3. Open the deployed Player Lab and include its URL in the submission.

.github/workflows/pages.yml | lines 20–27

```
20      steps:
21        - uses: actions/checkout@v4
22        - uses: actions/configure-pages@v5
23        - uses: actions/upload-pages-artifact@v3
24          with:
25            path: assignment3
26        - name: Deploy
27          id: deployment
```

Actual source excerpt, rendered with its original line numbers.

Published site URL: https://nurda07.github.io/Web-assignment/play.html
Repository: https://github.com/NURDA07/Web-assignment

11. Reflection and defense preparation

Challenges and solutions

State across steps: changing visible panels can lose user choices if markup is rebuilt without a state model. profileState holds the values, while onStepChange controls which panel is visible. Back therefore preserves input.

Keyboard access: clickable controls alone do not satisfy keyboard navigation. Navigation keydown handlers move focus, while the rating uses radio semantics, a single tab stop and arrow-key selection.

Browser audio restrictions: sound must follow an explicit user action. AudioContext creation/resume happens inside the click handler, playback is guarded, and ended callbacks clean up the audio nodes.

Safe rendering and file names: user names are inserted with textContent. Local resource paths were checked with exact letter case; the Azamat image reference was corrected so it also works on GitHub Pages.

What was learned

The same small set of JavaScript techniques can support different interactions: objects organize data, arrays hold collections, higher-order functions transform them, and event callbacks connect those values to DOM updates. Separating files and functions makes each feature easier to explain and change.

Suggested defense demonstration

1. Filter to Sandbox and explain switch, filter, map and getRating().
2. Change the gallery scene and point out src and alt.
3. Choose a star and demonstrate arrow keys in both the rating and navigation.
4. Complete the three profile steps, use Back, and reset. Explain the callback argument.
5. Play the chime and launch a quest. Explain the disabled guard and reduced-motion condition.
6. Show the separate source files and the deployment URL.

Submission checklist

The project ZIP contains HTML, CSS, JavaScript, images, README.md, DEFENSE.md, this PDF and its screenshot evidence. Each team member must submit and defend the work. Check the course LMS for the deadline; no deadline is inferred here.

Local launch from the extracted root:
python3 -m http.server 8000 --directory assignment3
Open http://localhost:8000/play.html. There is no install or build command.

All registration, contact, newsletter and player-profile flows remain educational demos. They do not send data, create accounts or subscriptions, or persist personal information.